FINAL REPORT: Knowledge Graph Construction, Embedding, and RAG System

1. Introduction

This project focuses on building and improving a **Knowledge Graph (KG)** using real-world data. The work is divided into three main stages:

- Knowledge Base (KB) construction
- Knowledge Graph Embedding (KGE) and reasoning
- SPARQL-based Retrieval-Augmented Generation (RAG)

The goal is to create a structured system that can store, learn, and answer questions using data efficiently.

2. Knowledge Base Construction (Lab 4)

2.1 Data Collection

Data was collected from the **OpenAlex API** using the keyword *"physics"*.

- Total authors: 1000
- Data includes: name, works count, citations, and research fields

2.2 RDF Graph Creation

The knowledge base was created using **RDFLib** in Python.

- Each author → unique URI
- Example: ex:Author_Name

Triples created:

- Type → Researcher
- Works count
- Field of work

Total triples: **3943**

2.3 Entity Linking

Entities were matched with **Wikidata**.

- Linked entities: 800

- Example:
    - Y. Bai → Wikidata ID
    - John Ellis → Wikidata ID

If no match:

- Entity stays local

2.4 Data Cleaning

- Removed missing values
- Limited concepts per author
- Avoided duplicate triples

2.5 Result

- Structured KB created
- Ready for expansion and learning

3. Knowledge Graph Embedding & Reasoning (Lab 5)

3.1 Ontology Reasoning

Ontology was built using **Owlready2**.

Classes:

- Person, Parent, Child, Father, Mother

Rule used:

- If age > 60 → OldPerson

Example:

- John (age 70) → OldPerson

Rule-based reasoning is:

- Accurate
- Easy to understand
- But not scalable

3.2 Knowledge Graph Preparation

- Total triples: 50,593
- After cleaning: 28,675
- Entities: 16,280
- Relations: 52

Dataset split:

- Train: 80%
- Validation: 10%
- Test: 10%

3.3 Embedding Models

Two models were used:

1. TransE

- Simple model
- Works for basic relations

2. DistMult

- Better for symmetric relations
- Performs better than TransE

3.4 Results

Model	**MRR**	**Hits@10**
TransE	Low	Low
DistMulT	Better	Better

Larger dataset improves performance:

- 20K → lower accuracy
- 50K → better accuracy

3.5 Key Insight

- Data quality is very important
- Embeddings give approximate answers
- Rules give exact answers

4. SPARQL-Based RAG System (Lab 6)

4.1 Concept

RAG combines:

- Knowledge Graph (data)
- Language Model (TinyLlama)

Goal:

- Convert question → SPARQL query → Answer

4.2 System Workflow

1. User question
2. Generate SPARQL query
3. Query RDF graph
4. Return answer
5. If fail → fallback to LLM

4.3 Example Query

```
SELECT ?entity ?occupation WHERE {
  ?entity <P106> ?occupation .
}
LIMIT 5
```

4.4 Results

Feature	Baseline LLM	RAG System
Accuracy	Low	High
Reliability	Poor	Good
Data usage	No	Yes

RAG gives:

- Correct answers
- Structured results

4.5 Challenges

- Entity names vs IDs mismatch
- Small model limitations
- Query generation errors

5. Overall Comparison

Method	Accuracy	Scalability	Interpretability
Rule-Based	High	Low	High
Embedding	Medium	High	Low
RAG System	High	High	Medium

6. Challenges Faced

- API rate limits
- Entity matching errors
- Noise in data
- Weak clustering in embeddings
- Limited computational power

7. Conclusion

This project successfully demonstrates:

- Building a Knowledge Base from real data

- Applying reasoning using rules
- Training embedding models
- Improving answers using RAG

Final Insight:

- Knowledge Graph + RAG = Best performance
- LLM alone is not enough
- Structured data improves accuracy